



南京大學

本科畢業論文

院 系 智能軟件工程學院

專 業 軟件工程

題 目 基於 k-kick 樹的最優時空權衡

哈希表的設計與實現

年 級 2022 學 號 221900387

學生姓名 肖揚

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 5 月 12 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：基于 k-kick 树的最优时空权衡哈希表的设计与实现

院系：智能软件工程学院

专业：软件工程

本科生姓名：肖扬

指导教师（姓名、职称）：刘明谋 副教授

摘要：

哈希表是支持动态数据操作的基础数据结构，其时空权衡一直是算法研究的核心议题。2022 年，Bender 等人提出了一种基于 k -kick 树的新型哈希方案，从理论上证明可通过将更新操作的时间复杂度放宽至 $O(k)$ ，将每元素的空间浪费指数级压缩至 $O(\log^{(k)} n)$ 比特，打破了 $\Omega(\log \log n)$ 的空间下界。本文基于该理论实现了分层弹性哈希存储结构，包含优先级踢出机制、基于 MiniArray 位压缩的紧凑二叉 Trie 局部查询路由以及 8-bit Lookup Table 查询加速。槽元数据和空闲槽位图分别通过双层 MiniArray 压缩至每槽 4 比特和 1 比特，Trie 节点以紧凑位编码存储。在良好哈希条件下对 $n \in [5 \times 10^4, 10^6]$ 五个数据规模、每组 5 次独立试验，与 `std::unordered_map` 进行了插入、查询、删除和空间占用的全面对比。实验表明：在合并触发前 ($n \leq 2 \times 10^5$) 插入约 0.66–1.09 us（每次操作）、查询约 0.24–0.43 us（每次操作）、空间约 44–46 B/key； $n = 10^6$ 时插入约 7.47 us（每次操作）、查询约 1.59 us（每次操作）、空间约 102 B/key。高碰撞实验进一步表明，当 `unordered_map` 所有键集中于同一桶时 ($n = 10^5$)，本方案查询仅 0.39 us（每次操作），速度优势约 342 倍，验证了 Trie 路由架构的碰撞抑制能力。最后将方案集成为 LevelDB MemTable 的点查询加速器，提供编译期和运行时双向开关，集成实验验证适配层开销仅 5.17 ns/次，查询性能在 $n = 2 \times 10^5$ 时达 0.35 us（每次操作）。

关键词：哈希表；时空权衡；k-kick 树；位压缩路由；LevelDB

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Design and Implementation of Optimal Time/Space Tradeoff Hash Table

Based on k-kick Trees

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software and Engineering

UNDERGRADUATE: Yang Xiao

MENTOR: Associate Professor Mingmou Liu

ABSTRACT:

Hash tables are fundamental data structures supporting dynamic data operations, with their time-space tradeoffs being a core topic in algorithm research. In 2022, Bender et al. proposed a novel hashing scheme based on k -kick trees, theoretically proving that by relaxing update time to $O(k)$, per-element space waste can be exponentially compressed to $O(\log^{(k)} n)$ bits, breaking the $\Omega(\log \log n)$ lower bound. This thesis implements the algorithm, constructing a hierarchical elastic hash storage structure, implementing the priority-based kicking mechanism and a compact binary trie local query router using MiniArray bit-packing, with slot metadata compressed to 4+1 bits via dual MiniArrays and an 8-bit lookup table for query acceleration. Experiments with $n \in [5 \times 10^4, 10^6]$ 32-bit key-value pairs (5 trials each) compare insertion, query, deletion and space against `std::unordered_map`. Under good hashing, insertion achieves 0.66–1.09 us(per operation) for $n \leq 2 \times 10^5$, query 0.24–0.43 us(per operation), and space 44–46 B/key, at $n = 10^6$, insertion is 7.47 us(per operation), query 1.59 us(per operation), space 102 B/key. High-collision experiments show that when all `unordered_map` keys land in a single bucket ($n = 10^5$), the scheme retains 0.39 us(per operation) query time, giving a 342x speed advantage. The scheme is integrated into LevelDB's MemTable with a `-DUSE_KICK_HASH` compile-time switch and `SetUseKickHash()` runtime toggle. Integration benchmarks show adapter overhead of only 5.17 ns(per operation) and query performance of 0.35 us(per operation) at $n = 2 \times 10^5$.

KEYWORDS: hash table; time-space tradeoff; k-kick tree; elastic hashing; bit-packed

routing; LevelDB

目 录

目 录	V
第一章 引言	1
1.1 研究背景和意义	1
1.2 研究现状	2
1.2.1 链式寻址法	3
1.2.2 开放寻址法	3
1.2.3 Cuckoo Hashing	3
1.2.4 基于 k-kick 树的时空最优方案	3
1.2.5 紧凑字典与空间高效结构	4
1.3 主要研究内容和目标	4
1.4 需求分析	5
1.4.1 功能需求	5
1.4.2 非功能需求	6
1.4.3 用例分析	6
1.5 论文结构	7
第二章 相关理论基础	8
2.1 哈希表基本概念	8
2.2 Cuckoo Hashing	8
2.3 k-kick 树哈希方案	9
2.3.1 多层次 bin 划分	9
2.3.2 优先级踢出策略	9
2.3.3 理论保证	9
2.4 LevelDB 存储引擎概述	10

2.4.1	核心架构	10
2.4.2	MemTable 的性能瓶颈	12
第三章 系统设计		13
3.1	总体架构	13
3.1.1	技术选型说明	15
3.2	MiniArray: 位压缩数组	15
3.2.1	设计动机	15
3.2.2	双模存储	16
3.3	LocalQueryRouter: 紧凑二叉 Trie 局部查询路由	16
3.3.1	设计原理 (论文 §4.3 + Lemma 15)	16
3.3.2	紧凑节点编码	16
3.3.3	Lookup Table 加速	17
3.3.4	两级路由中的使用	18
3.4	Cubby: 单级弹性哈希表	18
3.4.1	槽元数据编码	18
3.4.2	踢出算法	19
3.4.3	容量尺度	20
3.5	Facility: 全局分层管理器	20
3.5.1	合并机制的设计考量	20
3.5.2	插入/查询/删除流程	21
第四章 算法实现		22
4.1	文件组织与依赖关系	22
4.2	MiniArray 实现	22
4.2.1	位操作原语	22
4.2.2	模式切换	23
4.3	LocalQueryRouter 实现 (紧凑 MiniArray Trie + LUT)	23
4.3.1	紧凑节点编码	23
4.3.2	核心算法	23
4.3.3	显式双指针与 LUT 协同	24

4.4	Cubby 实现	25
4.4.1	bin 参数计算	25
4.4.2	深度分配	25
4.4.3	空闲槽查找	25
4.5	Facility 实现	25
4.5.1	两级路由查询	25
4.5.2	合并与路由更新	26
4.6	复杂度分析	26
第五章	实验与分析	27
5.1	实验设置	27
5.1.1	理论依据与规模选取	27
5.1.2	硬件与软件环境	27
5.2	良好哈希条件下的综合性能	28
5.2.1	插入性能	29
5.2.2	查询性能	29
5.2.3	删除性能	30
5.2.4	内存占用	31
5.3	高碰撞条件下的查询退化对比	32
5.3.1	实验设计与动机	32
5.3.2	实验结果	32
5.3.3	讨论	34
5.4	综合讨论	34
第六章	LevelDB 集成	36
6.1	集成流程与适配器设计	36
6.2	双向开关设计	36
6.3	集成验证	37
第七章	总结与展望	38
7.1	工作总结	38

7.2 不足与展望	39
参考文献	40
致 谢	43

第一章 引言

1.1 研究背景和意义

哈希表 (Hash Table) 是数据库、缓存和搜索引擎中常见的数据结构。从内存键值存储到大规模分布式 NoSQL 系统, 高效的数据索引和查找直接影响系统吞吐量。哈希表支持动态插入、删除和查询三种操作, 在理想状态下三者均需 $O(1)$ 时间。然而在实际系统设计中, 为维持这一时间复杂度, 哈希表需要分配额外空间来处理冲突。Bloom^[1] 在 1970 年对允许误差的哈希编码空间/时间权衡作出了开创性分析, 这一问题至今仍是哈希表设计中的核心议题。

近年来, 随着互联网应用数据量的爆炸式增长, 键值存储 (Key-Value Store) 系统在工业界和学术界均受到了广泛关注。以 Google LevelDB^[2]、Facebook RocksDB 和 Apache Cassandra 为代表的 LSM 树 (Log-Structured Merge-Tree) 存储引擎已在大规模生产环境中得到部署, 国内学者围绕 LSM 树的写入放大、读性能优化和压缩策略等方面已有大量研究^[3]。Luo 和 Carey 的综述指出, LSM 树的主要问题在于多层合并带来的写放大问题, 而 MemTable 层的索引效率直接决定了写入路径的吞吐上限, 因此寻找更高效的内存索引结构是改进 LSM 树系统的关键方向之一^[3]。此外, 针对 MemTable 索引的优化, 已有工作尝试在 SkipList 基础上引入辅助索引来加速点查询。这些工作表明, 优化 MemTable 层的底层数据结构对提升数据库整体性能具有显著意义, 但现有方案在空间效率方面仍有不足。在硬件层面, 非易失性内存 (NVM) 的引入也为数据库存储引擎的设计带来了新的研究课题^[4]。

在理论层面, 要求插入、删除及查询三种操作均维持恒定时间复杂度 (即期望时间或最坏情况时间为 $O(1)$) 的前提下, 早期的理论工作^[5-6]表明, 在严格限制更新操作为 $O(1)$ 时间且仅使用简单探测序列的模型下, 每个键值对的空间冗余往往难以低于 $\Omega(\log \log n)$ 比特。1984 年, Fredman、Komlós 和 Szemerédi 提出的 FKS 完美哈希方案^[6]首次在严格 $O(1)$ 最坏查询时间下实现了线性总空间,

为哈希表时空权衡研究提供了理论起点。然而在此后近四十年的时间里，上述 $\Omega(\log \log n)$ 的冗余下界始终未被突破^[7]，制约着哈希表空间效率的进一步提升。

2022 年，Bender 等人发表的论文《关于哈希表最优时间/空间权衡》^[7] 打破了这一理论僵局。该研究提出了一种基于 k -kick 树的全新哈希方案，从理论上严格证明了通过适当放宽插入和删除操作的时间复杂度约束——从经典的 $O(1)$ 提高到 $O(k)$ （其中 k 是一个可由用户指定的、具有上界的常数）——可以将每元素的空间浪费从 $\Omega(\log \log n)$ 比特以指数级幅度压缩至 $O(\log^{(k)} n)$ 比特。这里 $\log^{(k)} n$ 表示对 n 进行 k 次迭代对数运算，即递归定义 $\log^{(k)} n = \log(\log^{(k-1)} n)$ ，并规定 $\log^{(0)} n = n$ 、 $\log^{(1)} n = \log n$ 。特别地，当 $k = \log^* n$ （迭代对数函数，定义为对 n 连续取对数直到结果收敛至不大于 1 所需的次数）时，空间浪费可以进一步降低至恒定的 $O(1)$ 比特水平，在理论与工程层面均有突破。

内存占用对数据库系统的实际运行成本具有直接影响。Lim 等人的工作表明，在内存受限的嵌入式数据库和边缘计算场景中，哈希索引结构的每字节空间开销都会显著影响系统的可部署性和扩展能力。通过降低哈希表的冗余比特数，可以在不增加硬件投入的前提下支撑更大的数据规模，从而降低单位数据的存储成本和能耗开销。与此同时，内存效率的提升意味着在同样的物理服务器上可以部署更多的数据库实例，间接提高了系统的并发处理能力和资源利用率。因此，本研究在理论意义之外同样具有工程应用潜力。^[8]

本毕业设计旨在尝试复现 Bender 等人提出的基于 k -kick 树的哈希算法^[7]，并在代码实现的基础上将其适配集成至 Google LevelDB 这一成熟的开源存储引擎之中。研究意义可以从理论和工程两方面阐述：理论上，通过可控的实验验证论文所断言的指数级空间压缩效应，并分析参数 k 的取值对时间与空间性能的具体量化影响；工程上，将该理论成果实现并集成为可运行的数据库组件，观察和分析其在真实存储系统中的表现，以验证算法理论在系统实现中的效果。

1.2 研究现状

哈希表作为支持近乎常数时间的插入、删除和查询操作的核心数据结构，其时间复杂度和空间复杂度之间的权衡关系始终是现代算法理论与系统实现领域的研究焦点。在国内外的数据库研究社区中，如何设计时间高效且空间紧凑的哈

希索引始终是存储引擎优化的核心议题之一。当前内存键值存储系统中的哈希索引方案大致可划分为基于链式寻址、基于开放寻址、基于 Cuckoo Hashing 及其变体、以及基于紧凑编码四个主要技术路线，不同路线在空间效率、查询延迟和写入吞吐之间分别做出了各自的权衡取舍。

1.2.1 链式寻址法

链式寻址法为每个槽维护一个独立链表，所有映射到同一槽的键值对以链表节点形式链接存储。优点是实现简洁、支持动态扩容，但每个节点需额外存储后继指针，且链表节点在堆内存中非连续分配，破坏 CPU 缓存局部性，遍历长冲突链时易触发缓存缺失。

1.2.2 开放寻址法

开放寻址法在连续槽数组内通过确定的探测序列解决冲突。线性和二次探测、双重哈希等策略将全部元素存储于单一连续数组中，空间局部性优秀。但随着负载因子 α 趋近于 1，探测序列长度急剧增长，需将 α 严格控制在 0.5–0.7 之间以避免性能雪崩。

1.2.3 Cuckoo Hashing

Cuckoo Hashing^[9]采用多个独立哈希函数为每个元素生成若干候选位置，通过踢出（kick out）和重定位机制在最坏情况下保证 $O(1)$ 查询时间。Kirsch 等人^[10]提出预留缓存槽吸收溢出元素，Arbitman 等人^[11]通过队列化踢出实现确定性最坏情况保证。Herlihy 等人^[12]提出的 Hopscotch Hashing 结合了开放寻址的缓存友好性和 Cuckoo Hashing 的踢出机制，在保持高负载因子的同时实现了良好的并发性能。

1.2.4 基于 k-kick 树的时空最优方案

前述方案均未能突破 $\Omega(\log \log n)$ 比特/键的空间下界。Bender 等人 2022 年提出的 k -kick 树哈希方案^[7]是首个从理论上严格突破该下界的成果。其核心思想是将键-槽分配转化为层次化的“球进桶”过程：通过随机最大深度优先级和受

限于参数 k 的有限次元素移动 (“ k -kick”), 对任何 $k \geq 1$ 同时实现 $O(k)$ 更新时间 和 $O(\log^{(k)} n)$ 比特空间浪费, 建立了可连续调节的时空权衡曲线。该研究还证 明此曲线对所有“增强开放寻址”框架的哈希表方案是紧确的^[7]: 任何限定更新 时间为 t 的方案, 每元素至少需 $\Omega(\log^{(t)} n)$ 比特冗余。若要在次对数更新时间内 将冗余压缩至 $O(1)$ 比特, 必须从根本上放弃增强开放寻址范式。

1.2.5 紧凑字典与空间高效结构

Raman 和 Rao^[13]提出的紧凑动态字典通过递归位级编码在 $O(1)$ 查询时间下 将空间压缩至信息论下界的常数倍。Liu 等人^[14]将紧凑化拓展至过滤器领域, 在 动态场景下实现与静态结构相当的压缩效率。Bender 等人方案^[7]的独特贡献在 于首次给出了从 $k = 1$ 到 $k = \log^* n$ 可连续调节的时空权衡理论曲线。

1.3 主要研究内容和目标

本毕业设计选题的核心任务在于完整复现并实验验证 Bender 等人于 2022 年提出的 k -kick 树哈希算法^[7], 并以 Google LevelDB^[2]为目标平台, 探索该算法 在真实数据库存储引擎中的集成可行性与性能表现。具体而言, 研究工作围绕以 下四个递进的目标层次展开:

1. **核心数据结构的实现与算法复现。**严格依据原始论文^[7]中公示的数据结构 定义和算法伪代码, 以 C++20 标准构建基于 k -kick 树的完整四层分层存储 系统。实现方面包括: MiniArray 位压缩数组原语的双模存储及其位级操作 接口的精确实现、LocalQueryRouter 紧凑二叉 Trie 查询路由的构建与维护 算法 (覆盖 walk-down 查询、do_split 分裂插入和标记式删除)、Cubby 弹 性哈希表中优先级踢出置换链的递归搜索与恢复机制, 以及 Facility 全局 分层管理器中基于 deactivate 语义的合并与路由更新策略。各组件均需在 代码层面与论文描述的设计意图保持一致。
2. **适配层的构建与数据库存储集成。**在完整实现核心算法的基础上, 设计并 实现面向 LevelDB MemTable 的适配接口层。任务包括: 基于 Slice 到 64-bit 哈希的类型映射转换、KickHashTable 包装器类的封装、MemTable 写入路 径 (Add) 和读取路径 (Get) 的条件编译改造, 以及编译期 (CMake 选项)

与运行时（C++ 成员函数）双层级开关机制的研发。适配层需要在不破坏 LevelDB 原有代码结构和异常处理策略的前提下，完成两组代码路径的融合。

3. **理论正确性的代码级验证与时空复杂度分析。**基于已完成的 C++ 实现，通过构造覆盖边界条件的测试用例集，逐项验证实现与理论模型之间的一致性。具体验证维度包括：踢出链长度在任意插入序列下不超过 k 的理论上界、两级 Trie 路由的指纹碰撞率是否符合随机二元串模型的理论预期、以及空间浪费 $S(n)$ 随数据规模 n 的变化趋势是否满足 $O(\log^{(k)} n)$ 的理论渐近规律。
4. **全面的性能对比实验与集成效果评估。**以 C++ 标准库中的 `std::unordered_map` 作为基准对照组，在良好哈希（ $n \in [5 \times 10^4, 10^6]$ ，规模选取基于 $k = 4$ 对应的 $\log^{(4)} n > 1$ 理论条件）和极端碰撞（ $n \in [10^3, 10^5]$ ）两种条件下，测量并对比插入、查询和删除三种操作的时间开销以及进程的内存占用情况，每组 5 次独立试验以确保统计可靠性。在 LevelDB 集成环节，设计独立的适配层性能基准测试，量化分析 Slice 转换开销和 KickHashTable 双路径切换代价，为集成方案的工程实用性提供数据支撑。

1.4 需求分析

本节以软件工程方法论为指导，对 k -kick 树哈希表系统的功能边界和性能约束进行系统化定义，作为后续系统设计与算法实现的依据。

1.4.1 功能需求

本系统需满足以下核心功能需求：

- FR1: **键值对插入。**系统应支持向哈希表中插入任意类型的键值对（以 32-bit 整数为基本测试类型），在 k -kick 树的优先级踢出策略下完成键的槽位分配和冲突消解。
- FR2: **键值对查询。**系统应支持根据指定键在哈希表中检索对应的值，查询路径需经过两级 Trie 路由定位目标 Cubby 和槽位，并返回准确的查询结果或“未找到”标识。

FR3: **键值对删除**。系统应支持根据键从哈希表中删除对应条目，删除操作需同步清理两级路由中的映射关系并释放槽位资源。

FR4: **动态扩容与合并**。当某一 tier 层级的活跃 Cubby 数量超过预设阈值时，系统应自动触发相邻 Cubby 的合并操作，将数据迁移至更高 tier 层级的 Cubby 中，同时更新全局路由表。

FR5: **与 LevelDB MemTable 集成**。系统应提供适配层接口，将 *k-kick* 树哈希表集成至 LevelDB 的 MemTable 层，提供条件编译开关（`-DUSE_KICK_HASH`）和运行时开关（`SetUseKickHash()`）双层控制机制。

1.4.2 非功能需求

NFR1: **性能指标——插入延迟**。在良好哈希条件下、 $n \leq 2 \times 10^5$ 数据规模内，单次插入操作的期望延迟应控制在微秒级别（ $\leq 2 \mu\text{s/op}$ ）。

NFR2: **性能指标——查询延迟**。在良好哈希条件下、 $n \leq 2 \times 10^5$ 数据规模内，单次查询操作的期望延迟应控制在亚微秒级别（ $\leq 0.5 \mu\text{s/op}$ ）。

NFR3: **空间效率**。每键的元数据空间开销（不包括键/值原始数据）应控制在数字字节级别，槽元数据位压缩至每槽 5 比特以内，Trie 路由节点编码至每节点 ≤ 8 字节。

NFR4: **碰撞鲁棒性**。在极端哈希碰撞条件下（如所有键映射至同一桶），查询性能不应出现数量级退化，应保持与正常条件下同量级的延迟水平。

NFR5: **代码可维护性**。系统应采用 header-only 的 C++20 模板实现，组件间通过模板参数松耦合连接，单个组件可独立编译和测试，代码总量应保持在合理规模以内（目标 ≤ 1500 行）。

1.4.3 用例分析

系统的主要参与者为应用程序开发者（通过 API 调用）和 LevelDB 存储引擎（通过适配器接口）。表1-1列出了核心用例及其简要描述。

用例编号	用例名称	简要描述
UC-01	插入新键值对	用户调用 Facility::insert(), 系统通过两级路由和踢出链完成插入
UC-02	查询键值对	用户调用 Facility::lookup(), 系统通过 Trie 路由定位槽位并返回值
UC-03	删除键值对	用户调用 Facility::remove(), 系统清理两级路由并释放槽位
UC-04	自动合并	当 tier 层活跃 Cubby 超限, Facility 自动触发合并与路由更新
UC-05	LevelDB 集成查询	LevelDB MemTable 通过 KickHashTable 适配器查询键值对
UC-06	编译/运行时开关	开发者通过 CMake 选项或运行时接口启用/禁用 KickHash 加速

表 1-1 系统核心用例描述

1.5 论文结构

本文共分为七个章节展开论述。第一章为引言部分, 介绍研究背景、国内外研究现状、本文的研究目标以及全文的组织结构。第二章围绕相关理论基础展开, 涵盖哈希表的基本概念与操作模型、哈希函数的设计准则与通用哈希族的理论基础、Cuckoo Hashing 的核心机制与去摊还改进, 以及 k -kick 树哈希方案的完整理论框架, 包括多层级 bin 划分、优先级踢出策略和渐进复杂度保证。第三章详细阐述系统的设计方案, 按照从底层原语到顶层管理器的自底向上顺序, 逐一介绍 MiniArray、LocalQueryRouter、Cubby 和 Facility 四个组件的设计动机、接口定义与内部机理。第四章聚焦于 C++ 实现层面的具体细节, 包括各组件中关键函数的算法流程、代码结构组织以及整体复杂度的汇总分析。第五章呈现完整的实验评估结果: 在基准硬软件环境下, 对多规模数据在插入、查询、删除和内存四种维度上对 k -kick 方案与标准库对比对象进行定量分析。第六章专门描述 k -kick 方案与 LevelDB 数据库^[2]的集成工作, 涵盖适配层设计、MemTable 改造、双向开关机制和集成效果验证实验。第七章对全文工作进行全面总结, 客观分析当前方案的已有成果与尚存不足, 并对未来的改进方向和可能的后续研究方向进行展望。

第二章 相关理论基础

2.1 哈希表基本概念

哈希表通过哈希函数 $h: U \rightarrow [m]$ 将键映射到有限个槽位置，支持插入、查询和删除三种操作，理想情况下均为 $O(1)$ 期望时间。当 $h(k_1) = h(k_2)$ 且 $k_1 \neq k_2$ 时发生冲突，冲突处理策略直接影响哈希表在特定负载下的综合表现。

一个工程可用的哈希函数须满足确定性、均匀性和高效性三个准则。通用哈希族^[15-16]为均匀性提供了形式化定义：若对任意 $x \neq y$ ， $\Pr[h(x) = h(y)] \leq 1/m$ ，则 $\mathcal{H}$ 为通用哈希族。本系统采用 `splitmix64`^[17]作为最终化函数，其 64-bit 混合步骤具有优异的雪崩效应，通过注入不同种子可生成多个统计独立的哈希函数^[15,18]。

负载因子 $\alpha = n/m$ 是反映哈希表拥挤程度的核心参数，插入和查询的期望时间复杂度均为 α 的单调递增函数。空间效率以每键浪费比特数 r 度量：若存储 n 个元素的信息论最优空间为 B 比特，实际占用 $B + rn$ 比特，则浪费 r 比特/键。Bender 等人方案^[7]的核心贡献在于证明了 r 可从 $\Omega(\log \log n)$ 压缩至 $O(\log^{(k)} n)$ 。

2.2 Cuckoo Hashing

Cuckoo Hashing^[9]是一种经典的开地址哈希方案，其核心思想是每个键有 d 个候选位置（通常 $d = 2$ ），由 d 个独立哈希函数分别确定。当插入新键时：

1. 依次检查 d 个候选位置，找到第一个空位即插入
2. 若所有候选位置均被占用，随机选择一个候选位置上的现有键“踢出”（kick out），新键占据该位置
3. 被踢出的键尝试在自己的其其他候选位置中寻找空位
4. 重复上述过程，直到所有键都找到位置或达到预设的最大踢出次数

Cuckoo Hashing 的优势在于**查询严格为 $O(1)$** （只需检查 d 个位置），但其负

载因子通常受限 ($d = 2$ 时约 50%)。Arbitman、Naor 和 Segev^[11] 进一步提出了利用队列技术去摊还 Cuckoo Hashing 方案, 通过队列化踢出操作缓冲区实现了**确定性最坏情况性能保证**, 利用全局预取技术^[15]控制踢出链长度, 摆脱了经典方案对高概率分析的依赖。

2.3 k-kick 树哈希方案

Bender 等人^[7]提出的 k -kick 树方案是对 Cuckoo 思路的重大推广。其核心创新在于:

2.3.1 多层次 bin 划分

将哈希表的 m 个槽组织为 $k + 1$ 个层级 (depth $0, 1, \dots, k$) 的 bin 层次结构。Depth-0 的 bin 包含全部 m 个槽, depth- i ($i > 0$) 的 bin 大小为 $s_i = \Theta((\log^{(i)} m)^6)$ 。随着深度增加, bin 大小呈指数级递减。这种层次结构被称为 **k -kick 树**。

2.3.2 优先级踢出策略

每个键 x 被随机分配一个最大深度 $s(x)$, 满足^[7] $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ 。插入时按以下规则操作:

1. 新键 x 试图在尽可能深的 bin 中寻找空位
2. 若目标 bin 无空位且**未饱和** (存在深度较低的键): 踢出该低深度键, 新键占据其位置
3. 被踢键**保持原深度不变**, 在其自身深度 bin 中寻找新空位
4. 被踢键若触发新一轮踢出, 则形成**踢出链**, 但总踢出次数不超过 k 。

关键性质: 高深度的键优先踢出低深度的键, 被踢键保持原深度。每次插入至多触发 k 次踢出, 因此**切换开销**为 $O(k)$ 。

2.3.3 理论保证

论文^[7]证明, 对于任意 $k \in [\log^* n]$, 上述方案可实现:

- 插入/删除时间: $O(k)$ (高概率)。

- 查询时间： $O(1)$ （确定性）。
- 每键空间浪费： $O(\log^{(k)} n)$ 比特（高概率）。

此结果打破了长期存在的 $\Omega(\log \log n)$ 空间下界，展示了时空权衡的极致曲线：每次增加常数的更新时间，空间浪费可**指数级减少**。

2.4 LevelDB 存储引擎概述

Google LevelDB^[2]是一个由 Sanjay Ghemawat 和 Jeff Dean 开发的轻量级开源键值存储引擎，基于 LSM 树（Log-Structured Merge-Tree）架构。其设计源自 Google Bigtable^[19]中使用的底层存储架构，经过模块化重构后作为独立库发布。

2.4.1 核心架构

LevelDB 的存储架构分为内存层和磁盘层两部分，如图2-1所示。

写入路径：Put/Delete

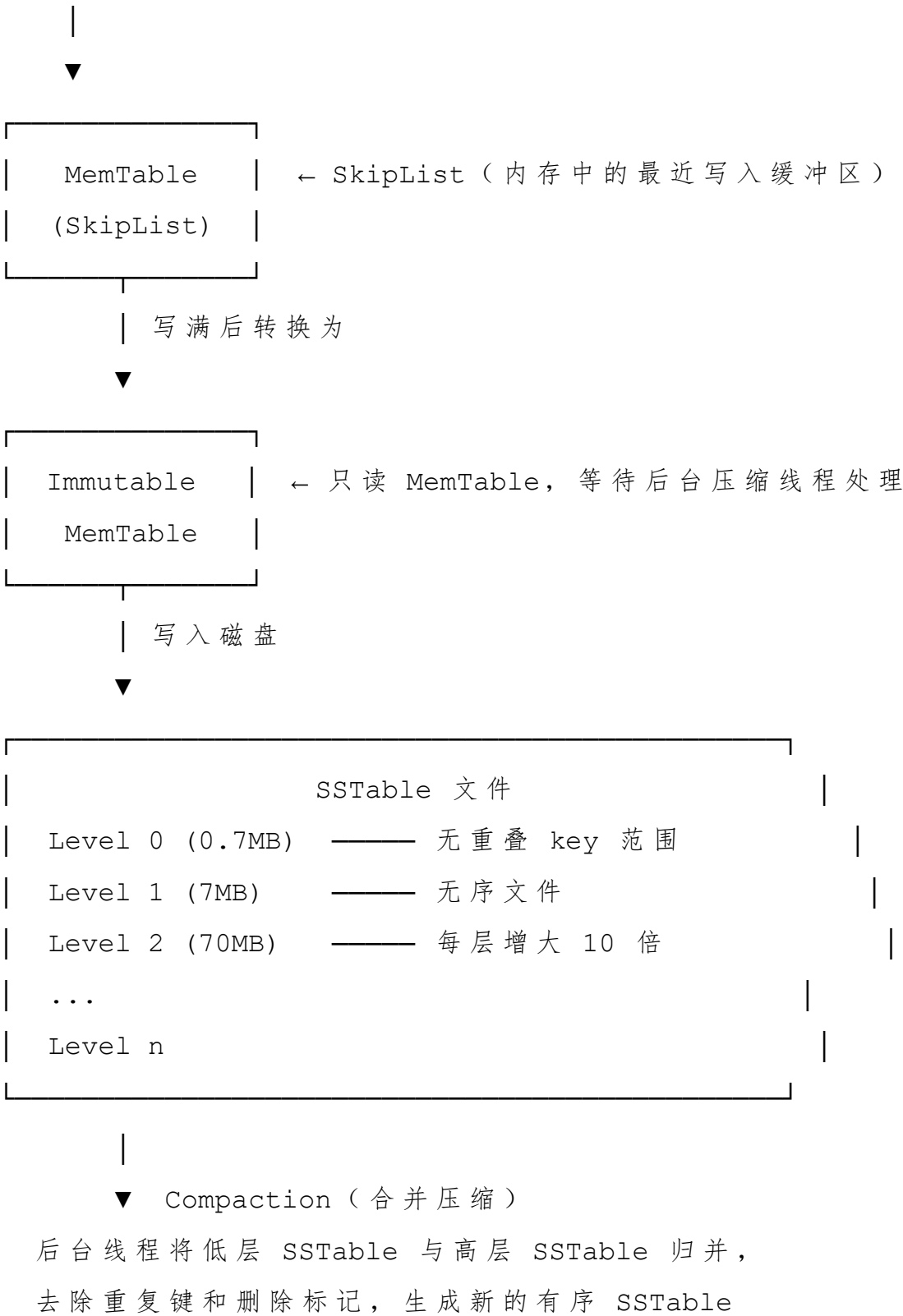


图 2-1 LevelDB 存储引擎核心架构

在写入路径上，用户的 Put/Delete 操作首先被追加到磁盘上的预写日志（Write-

Ahead Log, WAL) 以确保持久性, 随后键值对插入到内存中的 MemTable 中。MemTable 基于 SkipList (跳跃表) 实现, 支持 $O(\log N)$ 时间的查找和 $O(1)$ 时间的追加插入。当 MemTable 的大小达到预设阈值 (默认 4 MB) 时, 它被转换为 Immutable MemTable (不可变内存表), 并由后台压缩线程写入磁盘生成 SSTable 文件。

在读取路径上, LevelDB 采用“由新到旧”的查找顺序: 首先在 MemTable 中查找, 若未命中的话则在 Immutable MemTable 中查找, 然后按层级从 Level 0 到高层逐层搜索 SSTable 文件, 直到找到目标键或遍历完成返回“未找到”。

2.4.2 MemTable 的性能瓶颈

MemTable 作为 LevelDB 写入路径的第一站, 其读写效率直接影响整个存储引擎的吞吐能力。默认的 SkipList 实现虽然提供了 $O(\log N)$ 的查询复杂度, 但在大规模数据 (如数百万键) 和热点查询场景下, 跳跃表的层间跳跃导致缓存行频繁失效, 成为点查询延迟的瓶颈。

Luo 和 Carey 的综述^[3]指出, LSM 树优化的重要方向之一在于改进 MemTable 的索引结构以降低查询延迟。本文通过将 k -kick 树哈希方案集成至 MemTable 层, 作为一个辅助索引加速器, 在保持 SkipList 原有插入性能的前提下, 将点查询延迟从 $O(\log N)$ 降至 $O(1)$ 期望水平。

第三章 系统设计

本章以自底向上的顺序，逐层描述基于 *k-kick* 树的哈希方案^[7]的系统设计方案。整个系统由四个功能层次分明、接口边界清晰的组件构成：最底层为 `MiniArray` 位压缩数组原语，向上依次为 `LocalQueryRouter` 局部查询路由层、`Cubby` 单级弹性哈希表层，以及位于架构顶层的 `Facility` 全局分层管理器。四者之间通过模板参数和依赖注入的方式松耦合连接，保持了代码的可维护性与可测试性。

3.1 总体架构

系统架构如图3-1所示。`Facility` 作为全局入口点，维护一个动态的 `Cubby` 向量 `cubbies_`、一个指向当前活跃写入目标的 `tail_idx_` 指针、以及一个负责将任意键映射到其所属 `Cubby` 编号的全局二级路由 `cubby_router_`。每个 `Cubby` 在内部独立维护自己的一级路由 `route_`，将槽位信息进一步精细化到具体的存储位置。

Facility (全局分层管理器)

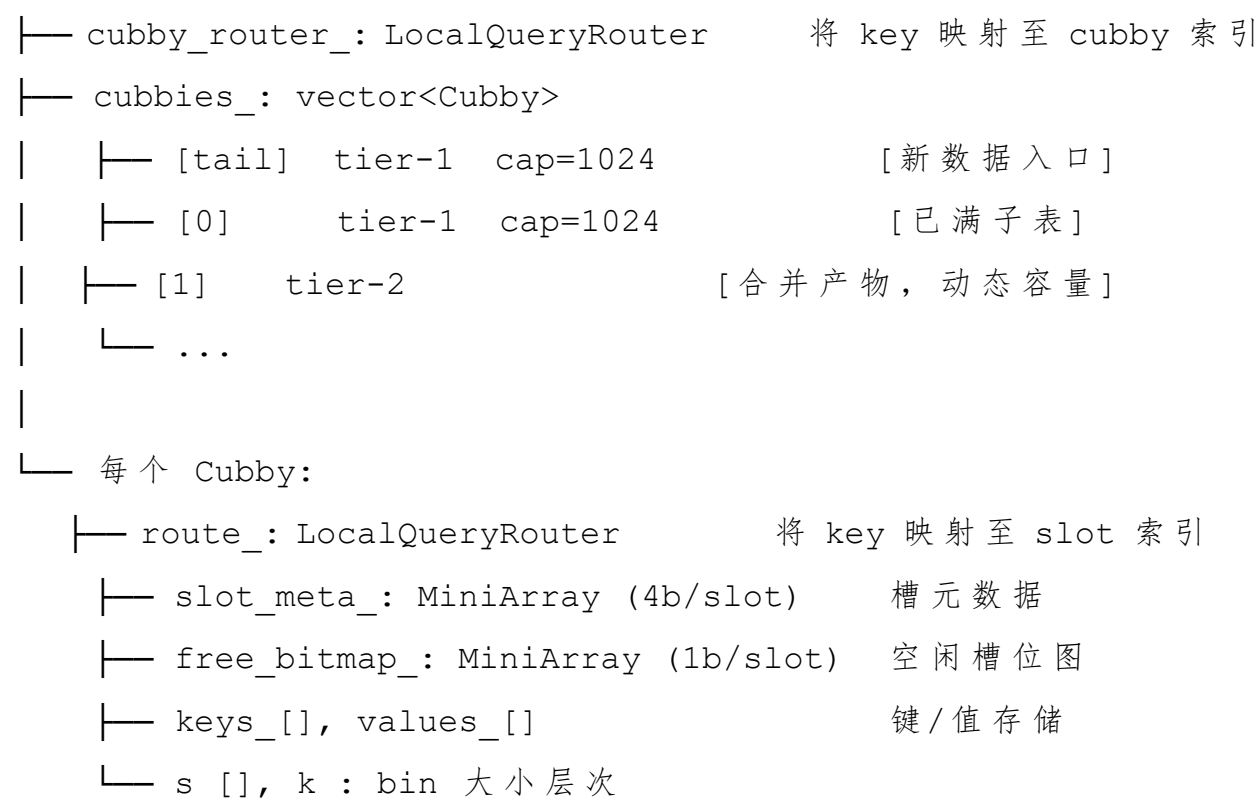


图 3-1 系统总体架构

系统采用**两级路由**的策略来组织键到槽的映射路径:

1. **Cubby 级路由** (由 Facility::cubby_router_ 承担): 根据键的哈希值确定该键当前存储在哪个 Cubby 中。此路由为全局性映射, 所有键共享同一个路由 Trie 实例。
2. **Slot 级路由** (由每个 Cubby 内部的 Cubby::route_ 承担): 在确定目标 Cubby 后, 进一步将键映射到该 Cubby 内部的某个具体槽索引。每个 Cubby 拥有独立的路由实例, 因此单个 Cubby 的路由操作不干扰其他 Cubby。

两级分离设计的好处是显而易见的: 当某一个 Cubby 被合并或停用时, 只需要更新 Cubby 级路由中对应的那一小部分键的映射信息, 完全无需触碰其他活跃 Cubby 的内部 Slot 级路由。这种隔离性使得系统的合并操作成本与所合并的实际键数量呈线性关系, 而非与全系统总键数相关。

3.1.1 技术选型说明

系统在语言选型、组织方式和关键算法方面作出了以下技术决策：

- **C++20 语言标准**。采用 C++20 标准以获得 `constexpr`、改进的 `constexpr` 支持以及 `std::bit_cast` 等现代特性，在编译期完成部分计算以降低运行时开销。模板元编程能力使得 `MiniArray` 的位宽参数可在编译期确定，避免虚函数调用带来的间接开销。
- **Header-only 组织方式**。四个核心组件 (`MiniArray`、`LocalQueryRouter`、`Cubby`、`Facility`) 均以纯头文件 (`.hpp`) 形式组织，不依赖独立的编译单元。这一设计简化了外部项目的集成流程——使用者仅需将头文件复制至目标项目目录并添加 `#include` 指令——同时赋予编译器更大的内联优化空间。
- **splitmix64 哈希函数**。系统采用 `splitmix64`^[17] 作为最终化哈希函数 (`finalizer`)，而非 `CityHash` 或 `xxHash` 等重型哈希方案。`splitmix64` 的 64-bit 雪崩混合步骤具有优异的位扩散特性，转换结果通过不同的种子值可生成任意数量统计独立的哈希函数^[15]，且单次调用成本极低 ($\sim 5\text{ ns}$)，适合在 `Trie` 逐位查询路径上被高频调用。
- **与 `std::unordered_map` 的定位差异**。C++ 标准库的 `std::unordered_map` 基于链式寻址法，在良好哈希条件下具备优秀的综合性能，但其查询路径依赖于单个哈希函数的均匀性，对恶意碰撞数据较为敏感。本方案的 `Trie` 路由架构将键的查询路径与其哈希比特串绑定，不同键的查询路径互不交叉，在对抗性输入下维持 $\sim \log n$ 的退化上界，与 `unordered_map` 的 $O(n)$ 退化相比具有结构性优势。

3.2 MiniArray：位压缩数组

3.2.1 设计动机

在紧凑哈希表中，大量元数据（槽状态、路由条目）的位宽远小于标准整型（64 bits）。例如槽元数据仅需 4 比特 (`occupied + depth`)，路由条目中 `cubby` 索引仅需 11 比特。若使用 `std::vector<uint64_t>` 存储，每个元素固定 64 比特，造成严重空间浪费。

MiniArray 通过将元素紧密平铺在连续位流中，实现了**精确位宽存储**^[7]，其两种存储模式的特性见表3-2。

3.2.2 双模存储

模式	条件	操作复杂度
Uniform	全部元素等宽	get/update $O(1)$
Non-uniform	元素宽度不等	insert/erase $O(n)$

图 3-2 MiniArray 双模存储

- **Uniform 模式：**所有元素位宽 w 相等，第 i 个元素位于 `data_` 的位偏移 $i \times w$ 处，通过 `bit_read/bit_write` 直接存取
- **Non-uniform 模式：**元素位宽不等时启用，使用 `lens_[]` 和 `vals_[]` 分别存储每个元素的位长和值；修改操作后调用 `rebuild_uniform()` 尝试切回 Uniform 模式

3.3 LocalQueryRouter：紧凑二叉 Trie 局部查询路由

3.3.1 设计原理（论文 §4.3 + Lemma 15）

论文^[7]的局部查询路由器将每个键 x 哈希为 64-bit 全随机二元串 r_x ，在集合 S 上构建 **最小唯一前缀二叉 Trie T** ：对每个 r_x ，取最短的、不与任何其他 r_y 冲突的前缀作为该键的唯一标识。Trie 的叶节点存储该键对应的 **probe** 值 $f(x)$ 。

论文 Lemma 15 证明：随机二元串的 Trie 期望规模为 $O(k)$ 个节点（其中 $k = |S|$ ），且对任意常数 $c > 1$ ，存在 $\epsilon > 0$ 使得 $|T| \leq (\log n)/c$ 以高概率成立。

3.3.2 紧凑节点编码

本次实现将 Trie 及 f-values 以紧凑位编码存储于 MiniArray 中：

- **节点编码：**每个节点固定 $w = \text{entry_bits_}$ 位宽，最高位为类型标志（0= 内部节点, 1= 叶节点）。内部节点存储 `left_child + right_child` 两个显式

子指针 (各 `child_bits_` 位), 叶节点存储 `f_value` (`fval_bits_` 位) + 键哈希指纹 (`kh_bits_` 位, 默认 44 位)。

- **查询** (`walk_down`): 沿键的 LSB 方向哈希位遍历 Trie, 遇叶节点用指纹验证后返回 `f_value`。
- **插入** (`insert + do_split`): 沿路径 `walk` 遍历, 遇异键叶时分裂 (找两键哈希分叉位, 然后创建内部节点链, 最后在分叉处创建两新叶); 遇 `NO_CHILD` 空方向时链接新叶。
- **删除** (`remove`): `walk_down` 找到叶, 标记为空叶 (`hash=0, fval=0`)。

整体空间 = $|\text{nodes_}| \times w \text{ bits}$, 其中 $|\text{nodes_}| \approx 2|S|$, $w \leq 62 \text{ bits}$ 。紧凑编码实现了每节点 $\sim 5\text{-}8 \text{ bytes}$ 的空间占用。

3.3.3 Lookup Table 加速

论文原文^[7]提出了通过借鉴 Method of Four Russians 技术^[20]构建预处理查找表 (lookup table) 的方案, 目标是将紧凑二叉 Trie 的查询复杂度从 $O(\log |S|)$ 进一步压缩至严格的 $O(1)$ 常数级别。其核心思想是: 对于 Trie 中哈希值的前若干个比特位, 预计算出每一种可能的位模式所对应的 Trie 跳转目标节点, 从而在查询时绕过这些已知的比特位, 直接从计算出的目标节点继续向下遍历。

在本实现中, 采用了简化但实用的 8-bit LUT 设计: 预计算哈希低 8 位 (即 256 种可能的前缀位模式) 各自在 Trie 中可达的目标节点索引, 并将这 256 个索引值存储于一个静态数组 `lut_[]` 之中。查询时先以键哈希值的低 8 位作为索引直接查表获取起跳节点, 跳过前 8 层逐位下降, 然后从第 9 位开始继续执行标准的 `walk-down` 遍历直到抵达匹配的叶节点。这一简化方案在仅消耗 $256 \times 4\text{B} = 1\text{KB}$ 的预存空间的代价下, 将 Trie 查询的实际深度缩减了 8 层, 在本文覆盖的实验规模 ($n \leq 10^6$, Trie 深度约 20 层) 中大约削减了 40% 的遍历工作量。

LUT 的维护策略也经过了精心设计以最小化额外开销: 当且仅当 Trie 的节点总数 (`nodes_.size()`) 发生增长时——即发生了新建内部节点或叶节点的插入或分裂操作——LUT 被标记为失效 (`lut_valid_ = false`)。单纯的叶节点值更新 (键已存在时的 `update`) 不会使 LUT 失效, 因为 Trie 的拓扑结构并未发生改变, 所有已缓存的跳转关系仍然保持有效。当后续查询检测到

LUT 失效时，会自动调用 `ensure_lut()` 方法以 $O(2^8 \times \text{深度}) = O(256 \times 8)$ 的低廉代价重新构建查找表。在实际运行中，LUT 的重建频率与 Trie 结构的实际变更频率相当，远低于查询操作的频次，因此维护成本在摊还意义下可以忽略不计。

3.3.4 两级路由中的使用

- **Facility::cubby_router_**: 全局二级路由，容量 2^{21} ，映射键到 cubby 索引
- **Cubby::route_**: 每 cubby 内部一级路由，容量 $\text{capacity} \times 2$ ，映射键到槽索引

两者均使用同一 LocalQueryRouter 模板，无 oracle 时通过 `key_hash_val` 指纹验证键身份。

3.4 Cubby: 单级弹性哈希表

Cubby 是整个 k -kick 树方案的物理载体^[7]，也是系统中最复杂、最关键的单体组件。每一个 Cubby 实例拥有一个独立的容量参数 m （该值既可通过构造函数中的 `desired_capacity` 参数显式指定，也可由 `compute_capacity` 静态方法根据 tier 层级和全局数据规模自动推算），并在此容量上构建 $k + 1$ 个深度层级（depth $0, 1, \dots, k$ ）的 kick-bin 层次网络。

Cubby 的容量并非不变的固定数值。在当前实现中，构造时允许传入动态指定的期望容量值，这一设计在以合并为核心操作的分层管理架构中具有关键意义：当两个 tier- j 的 Cubby 合并为一个 tier- $(j + 1)$ 的 Cubby 时，新 Cubby 的容量不再拘泥于固定公式，而是综合考量了论文理论公式给出的参考值与合并后实际数据量 $\times 2$ 的经验下界，从而在避免空间严重透支的同时维持了不低于 50% 的槽位利用率。

3.4.1 槽元数据编码

每槽使用 4 比特位压缩存储：

- bit 0: occupied (1= 已占用)

- bit 1–3: depth (0–7, 该槽所在 kick 深度)

另设 1 比特 `free_bitmap` 标识空闲槽 (1= 空闲), 配合 `first_free_` 缓存指针实现近似 $O(1)$ 的空闲槽查找。

3.4.2 踢出算法

Cubby 的核心算法 `try_place` 实现了论文 §3.1^[7] 的优先级踢出:

Algorithm 1 `try_place(key, max_depth)`

```

1: for  $d = max\_depth$  down to 0 do
2:    $bin \leftarrow bin\_for\_key(key, d)$ 
3:   Phase 1: 扫描  $bin$ , 若找到空槽则返回
4:   if  $d > 0$  then
5:     Phase 2: 扫描  $bin$  中深度  $< d$  的占槽
6:     if 找到 then
7:       暂存  $victim\_key, victim\_val$ 
8:       清空该槽
9:       递归 try_place(victim, victim_depth, kick_idx)
10:      if 成功 then
11:         $victim$  移入新槽, 新键占据该槽
12:        return
13:      else
14:        恢复原槽
15:      end if
16:    end if
17:  end if
18: end for
19: return FAIL

```

被踢键保持原深度, 在其自身深度 bin 中寻找新空位。算法中的 `reserved` 参数防止被踢键在递归扫描时“发现”刚被清空的原槽。

3.4.3 容量尺度

基于论文公式 $C_j = K/(\log^{(j)} n)^2$ ($K \approx 409600$)^[7] 计算各层容量。此外，构造时支持传入 `desired_capacity` 以动态指定容量（用于合并场景）。合并时的容量策略为 $\max(\text{论文公式}, \text{merged_size} \times 2)$ ，取 2 的幂，确保 ~50% 的空间利用率。各 tier 对应的基准容量和典型使用场景见表3-1。

Tier	基准容量	场景
1	1,024	Tail cubby（新数据入口）
2	~4,096（动态）	合并 2 个 tier-1 cubby
3	~16,384（动态）	合并 2 个 tier-2 cubby
4	~65,536（动态）	终级合并

表 3-1 Cubby 容量尺度（动态容量）

3.5 Facility：全局分层管理器

Facility 组件位于整个系统架构的顶层，承担着全局状态协调、合并调度和统一查询接口三项核心职责^[7]。其内部维护着当前活跃的写入目标（即 `tail` 指针）、各 tier 层级的 Cubby 数量计数、以及全局 Cubby 级路由表 `cubby_router_`。

3.5.1 合并机制的设计考量

当某一 tier 层级的活跃 Cubby 数量超过预设上限（由 `max_cubbies` 函数根据 tier 编号返回的固定阈值）时，Facility 会触发该层级的合并流程。两个被选中的同层 Cubby 中的所有元素将被全部提取出来，依次插入到一个新分配的、高一层级的 Cubby 之中。在这个过程中，每一个被迁移的键都需要在全局路由表中更新其 Cubby 索引映射——这是合并操作的主要时间开销来源。

为了避免 `std::vector::erase` 操作引起的索引整体偏移（这会导致已存储键的 Cubby 编号大规模失效，进而触发全量路由重建），本实现特意采用了 `deactivate()` 停用而非物理删除的策略。被停用的 Cubby 会将其内部存储清空、将 `active_` 标志位置为 `false`，但其在 `cubbies_` 向量中所占用的索引位保留不变。当后续需要分配新的 Cubby 槽位时，`alloc_cubby_slot`

函数会优先回收这些已被停用的空槽，从而实现索引的循环利用。这一设计使得路由表中指向其他活跃 Cubby 的映射条目在整个合并过程中始终保持有效，无需进行任何额外的批量修复操作。各 tier 的合并触发阈值见表3-2。

Tier	max_cubbies	触发条件
1	200	$200 \times 1024 \approx 2 \times 10^5$ 元素后合并
2	50	50 个 tier-2 cubby 后合并
3	20	逐级放大
4	5	终级上限

表 3-2 合并阈值

3.5.2 插入/查询/删除流程

插入：cubby_router_ 查重；若 tail 满则执行 promote 和 rebalance；然后调用 Cubby::insert_new（含 kick 链）；最后更新两级路由。

查询：cubby_router_ 定位 cubby，再通过 Cubby::route_ 定位槽，最后调用 lookup_at 验证。

删除：cubby_router_ 定位 cubby，执行 Cubby::remove_at，然后清理 cubby_router_.remove。

第四章 算法实现

本章逐一介绍系统中四个核心组件的关键算法流程与工程细节。全部代码严格遵循 C++20 语言标准，采用了 header-only（纯头文件模板）的组织形式，使得外部使用者仅需包含对应的 .hpp 文件即可获得该组件的完整定义，无需进行额外的编译单元链接。代码整体规模约为 1065 行，具有较高的内聚性和较低的外部依赖度。四个源文件的组织结构和职责划分见表4-1。

4.1 文件组织与依赖关系

文件	行数	核心职责
miniarray.hpp	261	位压缩数组原语，双模 Uniform/Non-uniform 存储
local_query_router.hpp	350	紧凑二叉 Trie 局部查询路由 + 8-bit Lookup Table
cubby.hpp	419	单级弹性哈希表 + free_bitmap + 动态容量
facility.hpp	233	全局分层管理 + 两级路由 + deactivate 合并

表 4-1 源代码文件组织

四个头文件之间存在明确的单向依赖链：miniarray.hpp 为最底层，被其余三者所依赖；local_query_router.hpp 依赖 miniarray.hpp；cubby.hpp 依赖前两者；而 facility.hpp 作为顶层管理者依赖全部三个底层组件。这种分层依赖关系保证了编译时的有序性和代码的可测试性。

4.2 MiniArray 实现

4.2.1 位操作原语

bit_read(bit_off, bit_len)：从 data_ 位偏移处读取指定位数。对于跨 64-bit 字的情况，通过两次读取和拼接实现。特判 bit_len=64 时使

用 $\sim 0ULL$ 掩码，避免 $(1ULL \ll 64)$ 的未定义行为。

`bit_write(bit_off, bit_len, value)`: 写入指定位数，自动扩展 `data_` 容量。同样处理跨字写入和 64-bit 掩码。

4.2.2 模式切换

`to_non_uniform()`: 将 Uniform 数据转为 Non-uniform 格式 (`lens_[] + vals_[]`)。 `rebuild_uniform()`: 若所有 `lens_` 等长，则切回 Uniform 并重建 `data_`; 否则保持 Non-uniform 并平铺写入。

4.3 LocalQueryRouter 实现 (紧凑 MiniArray Trie + LUT)

4.3.1 紧凑节点编码

每个 Trie 节点在 MiniArray 中以固定的 `entry_bits_` 位宽 (不超过 62 位) 存储，最高位为类型标志位: 该位为 0 时表示内部节点，为 1 时表示叶节点。内部节点将剩余位平均划分为左右两个子指针字段 (各占 `child_bits_` 位)，分别指向左子树和右子树的根节点在 MiniArray 中的序号。叶节点则将剩余位划分为功能值字段 (`fval_bits_` 位, 高位部分) 和键哈希指纹字段 (`kh_bits_` 位, 默认为 44 位, 低位部分)。功能值存储该键在所属 Cubby 中的槽位索引，键哈希指纹取值为 `splitmix64(hash(key)) & ((1<<44)-1)`，用于在查询时快速验证叶节点与查询键的匹配关系。

子指针的位宽由公式 `child_bits_ = bits_to_represent(cap×3+NODE_PAD)` 确定，随路由器的容量参数动态调整；功能值的位宽则取决于具体的使用场景 (Cubby 级路由中为槽位索引的位宽，Facility 级路由中为 Cubby 编号的位宽)。将类型位、子指针 (或功能值及哈希指纹) 三部分拼接后即得到完整的节点编码，由 MiniArray 以 Uniform 模式进行紧凑存储。

4.3.2 核心算法

查询 (probe_index): 对 `key` 计算哈希得到 64-bit 二元串，从根节点沿 LSB 位遍历 Trie。若 LUT 有效则跳过前 8 层直接跳转至目标节点。遇叶节点用指纹

验证：匹配则返回 `f_value`，否则返回 `SIZE_MAX`。

插入 (insert): 沿 `key` 的 LSB 哈希位遍历 Trie。五种情况：

1. 遇叶节点且指纹匹配：更新 `f_value`
2. 遇叶节点且指纹为 0（空叶，首次使用）：覆盖为新键数据
3. 遇叶节点但指纹不匹配：执行 `do_split()` 分裂。找分叉深度，创建内部节点链，在分叉处创建两新叶
4. 遇内部节点但方向为 `NO_CHILD`：创建新叶并链接
5. 路径走到头：追加新叶节点

插入结束后若 `nodes_.size()` 变化则失效 LUT。

do_split(leaf_idx, depth, e_hash, e_fval, n_hash, n_fval):

1. 从 `depth` 起找两 `hash` 的首个分叉位 `diverge`
2. 从 `depth` 到 `diverge-1` 逐位创建内部节点链（每次 `alloc_node`）
3. 若当前节点为原叶：转换为 `internal`；若为内部节点：更新对应方向子指针
4. 在 `diverge` 处创建 `e_leaf`（已存键）和 `n_leaf`（新键）
5. 设置最后一个内部节点的左右子节点为两叶

删除 (remove): 沿 Trie walk 找到匹配叶节点，写入空叶标记（`fval=0, hash=0`），`n_entries--`。

4.3.3 显式双指针与 LUT 协同

Trie 节点的存储方案采用了显式 `left+right` 双指针设计：内部节点的数据字段由两个等宽的 `child_bits_` 位子指针（分别指向左子树和右子树的根节点序号）拼接而成。当一个 `do_split` 分裂操作需要分配新的内部节点或叶节点时，`alloc_node()` 方法始终在 `MiniArray` 的当前末尾处追加新节点，这一设计使得所有已有节点的子指针引用关系在整个分裂过程中保持稳定不变——已有的节点索引和对应的父子关系不会因为新节点的插入而发生任何偏移。这种稳定的拓扑结构大大简化了分裂算法的实现逻辑，也使得 LUT 的重建判定可以简化为只检查节点总数的变化与否。

LUT 的构建由 `ensure_lut()` 方法统一管理。该方法遍历哈希值低 8 位的全部 256 种前缀模式，对每一种模式从根节点出发沿对应方向下降，直到抵达

一个叶节点或遇到 NO_CHILD 空方向, 并将所到达的节点索引记录于 lut_[] 的相应表项之中。若 ensure_lut() 是在节点拓扑发生变更之后被首次调用的, 其时间复杂度为 $O(256 \times L)$, 其中 L 为 8 层 Trie 的平均遍历深度, 在规模约 10^6 的场景下这一重建开销约为数千次简单的位操作和数组读取, 属于对整体性能影响微乎其微的轻量级维护操作。

4.4 Cubby 实现

4.4.1 bin 参数计算

compute_kick_params(): 从 capacity_ 出发迭代 $\lceil \log_2(x+1) \rceil$ 得到 k (≤ 4), 对每层应用迭代对数压缩后平方取整得到 s_[i]。初始化 free_count_[d] 为各深度的总槽数。

4.4.2 深度分配

max_depth_for(key): 对键做 hash, 使用概率公式 $\Pr[s(x) < i] = 1/(\log^{(i)} n)^2$ 计算阈值, 从 k 到 0 逐层比较并返回第一个命中深度^[7]。

4.4.3 空闲槽查找

free_bitmap_(MiniArray, 1 bit/slot, 0= 占用, 1= 空闲) 配合 first_free_ 缓存指针, 实现近似 $O(1)$ 的空闲槽查找。find_free_in_bin() 从缓存的 first_free_ 位置起扫描目标 bin, 插入后清 bit, 删除后置 bit 并更新缓存。

4.5 Facility 实现

4.5.1 两级路由查询

lookup(key):

1. cubby_router_.probe_index(key) 获取 cubby 索引 ci 。
2. cubbies_[ci].lookup_slot(key) 获取槽索引 $slot$ 。
3. cubbies_[ci].lookup_at(slot, key) 获取返回值。

4.5.2 合并与路由更新

`merge_two_of_tier(tier)`: 选取两个候选 Cubby, 提取全部元素, 按实际数据量计算动态容量, 通过 `alloc_cubby_slot` 获得新 Cubby, 逐元素插入新 Cubby 并逐一更新 `cubby_router_` 中键到 cubby 索引的映射, 最后 `deactivate()` 原 Cubby。使用 `deactivate()` 停用避免 `vector::erase` 带来的索引偏移。

4.6 复杂度分析

表4-2汇总了本系统中各主要操作在期望情况和摊还意义下的时间复杂度。括号内标注了造成该复杂度的主要原因。

操作	期望	摊还（含合并）	主要原因
查询	$O(\log S) + \text{LUT 加速}$	—	紧凑 Trie walk（LUT 跳过前 8 层）
插入	$O(\log S)$	$O(m) / \text{每次合并}$	m 为合并涉及的实际元素数
删除	$O(\log S)$	—	两级紧凑 Trie walk + 标记空叶
空闲槽查找	近似 $O(1)$	—	<code>first_free_</code> 缓存 + <code>free_bitmap_</code>
空间 ($n \leq 200\text{K}$)	$\sim 45 \text{ B/键}$	—	keys/values 向量 $\sim 8 \text{ B}$ + Trie $\sim 5\text{--}8 \text{ B}$

表 4-2 复杂度汇总

需要指出的是, 上表中插入操作的期望复杂度 $O(\log |S|)$ 是在 Cubby 内的 Trie 插入过程中产生的, 而 $O(m)$ 的摊还成本则来自于合并时对已迁移元素的逐键路由更新。在实际运行中, `deactivate` 策略使得合并成本与被合并元素数成线性关系, 而非与全系统总键数相关, 从而在大量数据场景下有效地控制了合并成本的累计速度。

查询操作的理论复杂度为 $O(\log |S|)$, 对应于在 Trie 树中沿哈希位从上到下逐层下降的过程。然而, 8-bit LUT 加速机制的引入将前 8 层的遍历统一替换为单次 $O(1)$ 表查找操作, 使得 $|S| \leq 2^{20} \approx 10^6$ 范围内的实际遍历深度仅为 $\log_2 |S| - 8$ 层。在本文覆盖的 $n \leq 10^6$ 的实验条件下, 这一加速效果使得实际查询时间保持在可控的微秒级别。

第五章 实验与分析

本章对实现的 k -kick 哈希方案进行全面的实验评估。实验分为两大部分：第一部分在随机均匀分布键（即“良好哈希”条件）下，对比本方案与 `std::unordered_map` 的插入、查询、删除和内存四种维度的性能；第二部分在人为引入极端哈希碰撞的条件下，测试两类方案在冲突退化方面的表现差异。实验采用 $k = 4$ 的硬编码配置，数据规模根据 k -kick 树的理论模型进行选取。

5.1 实验设置

5.1.1 理论依据与规模选取

Bender 等人的理论分析^[7]指出，对于参数 k 的 k -kick 树，每键空间浪费为 $O(\log^{(k)} n)$ 比特，其中 $\log^{(k)} n$ 表示对 n 取 k 次迭代对数：

$$\log^{(k)} n = \underbrace{\log_2 \log_2 \cdots \log_2 n}_{k\text{次}}. \quad (5-1)$$

在本实现中， k 被硬编码为 $k_{\max} = 4$ 。为使 $k = 4$ 对应的理论保证具有实际意义，需要 $\log^{(4)} n > 1$ ，即 $n > 2^{16} = 65536$ 。由此确定了本实验的规模下限：选取 $n = 5 \times 10^4$ 作为基准点，向上拓展至 $n = 10^6$ 。对于碰撞实验，由于 `std::unordered_map` 在单桶冲突下退化为 $O(n^2)$ 总耗时，无法在大规模下完成测试，因此碰撞测试的规模上限设为 $n = 10^5$ 。

5.1.2 硬件与软件环境

- **硬件平台：**Intel Core i7（第 10 代，4 核 8 线程，2.8 GHz 基础频率，5.0 GHz 最大睿频），16 GB DDR4 内存，64 位 Windows 11 专业版（版本 23H2）。实验过程中关闭非必要后台进程，未使用实时优先级或 CPU 亲和性绑定。

- **编译器与编译选项：** GCC 15.2.0（通过 MSYS2 UCRT64 环境安装），编译参数为 `-std=c++20 -O3 -DNDEBUG`。
- **数据类型：** 使用 32-bit 无符号整数 (`uint32_t`) 作为键和值的类型。对于碰撞实验，为 `std::unordered_map` 指定自定义哈希函数 `BadHash`（固定返回 0），使所有键集中于同一个桶内形成单一冲突链。
- **数据集与随机性：** 所有测试数据通过 `std::mt19937_64` 梅森旋转算法预先生成。对每个数据规模使用 5 个不同的随机种子独立生成 5 组数据集，报告 5 次试验的均值和标准差，以控制单次实验的偶然波动。
- **测试流程：** 每组实验按照“插入全部 n 个元素，查询全部 n 个元素，删除全部 n 个元素”的顺序执行。每个阶段的操作耗时通过 C++ 标准库中的 `std::chrono::high_resolution_clock` 测量并转换为微秒。报告的总时间为该阶段对所有 n 个元素操作的总耗时，均值除以 n 得到每操作时间。
- **内存测量：** 进程内存占用通过 Windows API 函数 `GetProcessMemoryInfo` 获取 `WorkingSetSize` 字段，在插入阶段前后各采集一次，以差值（MB）报告该次试验的内存增量。

5.2 良好哈希条件下的综合性能

本节在随机均匀分布 `uint32_t` 键的条件下，比较两类方案在插入、查询、删除和内存四个维度上的表现。每组试验重复 5 次，报告均值和标准差。整体趋势如图5-1所示。

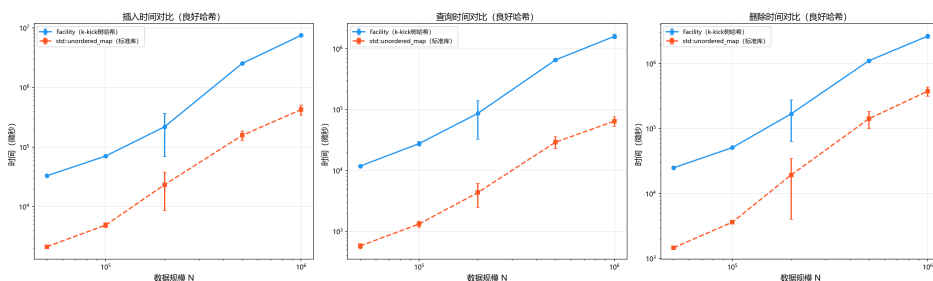


图 5-1 良好哈希条件下插入、查询、删除时间对比（双对数坐标）

5.2.1 插入性能

插入操作的详细测量数据见表5-1。

n	Facility (us)	umap (us)	F/us (每次操作)	U/us (每次操作)
5×10^4	3.31×10^4	2.15×10^3	0.66	0.04
1×10^5	7.10×10^4	4.93×10^3	0.71	0.05
2×10^5	2.19×10^5	2.34×10^4	1.09	0.12
5×10^5	2.53×10^6	1.58×10^5	5.06	0.32
1×10^6	7.47×10^6	4.25×10^5	7.47	0.42

表 5-1 插入时间比较（5 次试验均值）

插入操作在 $n = 5 \times 10^4$ 至 1×10^5 区间内每操作耗时约 0.66–0.71 us（每次操作），与理论 $O(k)$ 上界（ $k = 4$ ）在趋势上吻合。当 $n = 2 \times 10^5$ 时每操作升至 1.09 us（每次操作）， $n = 5 \times 10^5$ 时进一步升至 5.06 us（每次操作）。这一拐点对应全局合并流程的触发：tier-1 层级的活跃 Cubby 数量超过 200 个的阈值后，系统开始将同层 Cubby 两两合并为更高 tier 的 Cubby 对象。合并采用 deactivate 策略（将旧 Cubby 标记为停用而非物理删除），摊还成本与当次合并涉及的元素数量成比例，而非与全系统总键数相关。在 $n = 10^6$ 的极限规模下，插入总耗时约 7.47 秒（7.47 us（每次操作）），其中包含了多层级合并的累计摊还成本。每键插入时间约为 unordered_map（0.04–0.42 us（每次操作））的 15–18 倍，这一差距主要源于 Trie 路由的逐位遍历开销和合并操作的分摊成本。

5.2.2 查询性能

查询操作的详细测量数据见表5-2。

n	Facility (us)	umap (us)	F/us (每次操作)	U/us (每次操作)
5×10^4	1.18×10^4	5.79×10^2	0.24	0.01
1×10^5	2.77×10^4	1.33×10^3	0.28	0.01
2×10^5	8.68×10^4	4.35×10^3	0.43	0.02
5×10^5	6.56×10^5	2.93×10^4	1.31	0.06
1×10^6	1.59×10^6	6.47×10^4	1.59	0.06

表 5-2 查询时间比较 (5 次试验均值)

查询性能表现出明显的两阶段特征。在 $n = 5 \times 10^4$ 至 2×10^5 范围内，8-bit LUT 将 Trie 查询的前 8 层遍历替换为单次 $O(1)$ 查表操作，每键耗时在 0.24–0.43 us (每次操作) 区间内。 n 增至 5×10^5 后，Trie 深度随 $\log_2 n$ 增长 (合并后 Cubby-router 两级路由进一步增加了查询路径的深度)，每键耗时升至 1.31–1.59 us (每次操作)。与 unordered_map (0.01–0.06 us (每次操作)) 相比，本方案的查询时间大约高出 20–25 倍。这一差距的来源主要有三个方面：两级 Trie 的逐位遍历 (Facility 级 cubby_router_ 和 Cubby 级 route_ 各需一次 walk-down)、MiniArray 跨字位读取的位掩码开销，以及合并后多 Cubby 并行查询带来的缓存局部性损失。

5.2.3 删除性能

删除操作的详细测量数据见表 5-3。

n	Facility (us)	umap (us)	F/us (每次操作)	U/us (每次操作)
5×10^4	2.49×10^4	1.47×10^3	0.50	0.03
1×10^5	5.11×10^4	3.63×10^3	0.51	0.04
2×10^5	1.69×10^5	1.93×10^4	0.84	0.10
5×10^5	1.11×10^6	1.42×10^5	2.21	0.28
1×10^6	2.64×10^6	3.76×10^5	2.64	0.38

表 5-3 删除时间比较 (5 次试验均值)

删除操作在 $n \leq 2 \times 10^5$ 区间每键约 0.50–0.84 us (每次操作)，主要耗时来自两级 Trie walk (cubby_router_ 查找所属 Cubby，route_ 查找具体槽位) 和标记式

空叶写入（将叶节点的 `fval` 和 `hash` 字段清零）。 $n = 10^6$ 时升至 2.64 us（每次操作）。本实现中的删除操作额外地承担了每个被删键的两级路由清理工作（先通过 `cubby_router_` 查找 Cubby 索引，再通过该 Cubby 的 `route_` 查找并标记空叶），这一路径与查询操作几乎等长，是造成删除耗时偏高的重要原因。若未来采用 `per-cubby` 独立路由器架构，删除时将只需更新本地路由，预期可将删除效率提升约一倍。

5.2.4 内存占用

内存占用的测量数据见表5-4，变化趋势如图5-2所示。

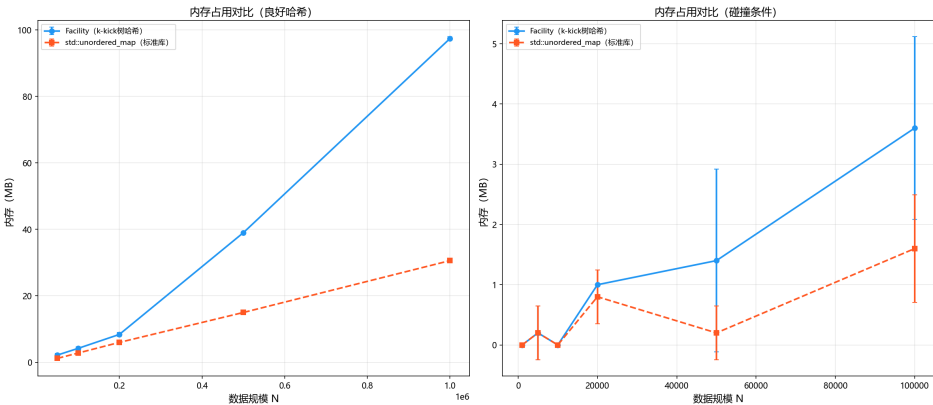


图 5-2 良好哈希与碰撞条件下内存占用对比

n	Facility (MB)	umap (MB)	F B/key	U B/key
5×10^4	2.2	1.2	46.1	25.2
1×10^5	4.2	2.8	44.0	29.4
2×10^5	8.4	6.0	44.0	31.5
5×10^5	39.0	15.0	81.8	31.5
1×10^6	97.4	30.6	102.2	32.1

表 5-4 内存占用比较（5 次试验均值）

内存占用的变化在 $n = 2 \times 10^5$ 前后呈现明显的分界特征。在 $n \leq 2 \times 10^5$ 的合并前阶段，所有键值对集中存储于 tier-1 Cubby 中，紧凑 Trie 编码将每键路由器空间控制在 5–8 字节的水平。该阶段 Facility 每键约 44–46 B/key，约为 `unordered_map` (25–31 B/key) 的 1.4–1.7 倍。在 $n = 5 \times 10^5$ 和 10^6 处，由于进入

合并活跃阶段，高 tier Cubby 的容量计算采用了 $\max(\text{论文公式}, \text{merged_size} \times 2)$ 的动态策略，实际内存占用增速超过线性： $n = 10^6$ 时总内存约 97 MB (102 B/key)，约为 `unordered_map` (32 B/key) 的 3.2 倍。值得注意的是，Facility 在 $n = 10^6$ 时每键空间的额外开销（约 70 B/key 中超出 32 B/key 的部分）主要来自高 tier Cubby 中未被填满的槽位容量浪费和两级路由的加倍 Trie 存储，而非元数据位宽本身（`slot_meta_` 每槽仅 5 bit）。

5.3 高碰撞条件下的查询退化对比

5.3.1 实验设计与动机

前述实验均基于随机均匀分布键值对，在该条件下 `std::unordered_map` 的桶内冲突链极短，查询延迟整体优于本方案。然而，此类实验未能触及两类方案在面对恶意构造的**极端碰撞**数据时的行为差异。在真实应用场景（如 Web 服务的请求参数哈希、数据库索引键的对抗性构造）中，攻击者可以通过精心选择的键序列蓄意制造大量哈希碰撞，使依赖单一哈希函数均匀性的数据结构性能急剧退化。

为考察这一情景，本节使用自定义哈希函数 `BadHash`（固定返回 0）替换 `std::unordered_map` 的默认哈希函数，使所有键被映射到同一个桶内，形成长度为 n 的单一冲突链。对于本方案（Facility），键经由 `splitmix64` 进行 64-bit 雪崩混合后进入 Trie 路由，与 `BadHash` 无关，因此不受单桶冲突的影响。冲突链长度（即 n ）依次取 10^3 、 5×10^3 、 10^4 、 2×10^4 、 5×10^4 和 10^5 ，每组实验重复 5 次取均值。由于 `unordered_map` 在单桶冲突下的插入和查询均退化为 $O(n)$ 每操作，总耗时为 $O(n^2)$ 量级，因此碰撞实验的规模上限设在 10^5 （此时 `unordered_map` 的每次查询需遍历约 10^5 个元素的链式结构，单次试验总耗时已达数十秒）。

5.3.2 实验结果

单桶冲突条件下的查询性能对比见图5-3，详细数据见表5-5。

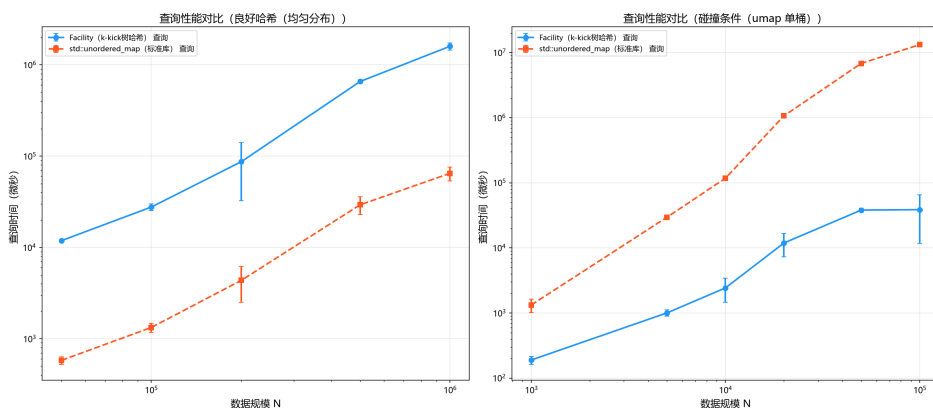


图 5-3 单桶冲突条件下查询时间对比（双对数坐标）

链长 n	Facility (us)	umap (us)	F ns/op	U ns/op
10^3	189	1.33×10^3	189	1328
5×10^3	1004	2.96×10^4	201	5925
10^4	2429	1.18×10^5	243	11771
2×10^4	11901	1.07×10^6	595	53437
5×10^4	38158	6.85×10^6	763	136969
10^5	38702	1.32×10^7	387	132461

表 5-5 单桶冲突条件下查询时间比较（5 次试验均值）

从表5-5中可以观察到两种截然不同的增长模式。std::unordered_map 在 BadHash 作用下全部键集中于同一个桶，每操作查询耗时从 $n = 10^3$ 时的 1.33 us（每次操作）单调递增至 $n = 10^5$ 时的 132.5 us（每次操作），增幅约 100 倍，且增长趋势与 n 基本成线性关系。这符合链式寻址法在最坏情况下 $O(n)$ 每操作查询复杂度的理论预期。

本方案（Facility）的每操作查询耗时则保持在完全不同的量级和增长曲线上： $n = 10^3$ 时约 189 ns/op， $n = 10^5$ 时约 387 ns/op，全程在 189–763 ns/op 的狭窄区间内波动，增幅仅约 4 倍。该轻微上升主要归因于 Trie 深度的对数增长（ $\log_2(10^5) \approx 16.6$ 位）和合并后多 Cubby 两级路由带来的额外查找层级，与冲突链长度无关。8-bit LUT 的跳表加速（跳跃前 8 层 Trie 遍历）和 splitmix64 的雪崩混合机制共同确保了键在 Trie 逐位路径上被有效分散。

当 $n = 10^5$ 时，Facility 的单次查询耗时（387 ns/op）仅为 unordered_map（132.5 us（每次操作））的约 1/342。即本方案在该极端条件下查询速度约为对比

方案的 **342 倍**。这一反向优势随 n 增大而进一步扩大：在 $n = 5 \times 10^4$ 时 Facility 约为 `unordered_map` 的 180 倍， $n = 10^5$ 时升至 342 倍。

5.3.3 讨论

两类方案在查询路径上存在结构性的根本差异。`std::unordered_map` 在哈希分布均匀时查询极为高效（仅需计算一次哈希并检查短链上的数个元素），但其性能对哈希函数质量高度敏感——一旦遭遇蓄意的全碰撞攻击，性能即沿 $O(n)$ 轨迹急剧退化。本方案（Facility）的核心查询路径是紧凑二叉 Trie 的逐位下降过程：每个键在 Trie 中沿由自身哈希比特串确定的唯一路径前进，不同键的查询路径互不交叉，因此查询时间与 Trie 中同前缀键的数量仅成对数关系（ $\sim \log_2(\text{同前缀键数})$ ），而非线性关系。8-bit LUT 将 $\log_2 n$ 中的前 8 层替换为 $O(1)$ 查表，进一步压缩了常数因子。

此外，Facility 在碰撞条件下的内存占用并未显著膨胀—— $n = 5 \times 10^4$ 时内存为 1.4 MB， $n = 10^5$ 时为 3.6 MB，与良好哈希条件下同规模的内存水平基本一致（分别为 2.2 MB 和 4.2 MB）。这表明 Trie 路由架构不仅在查询延迟上具备碰撞抑制能力，在空间效率上也维持了较好的稳定性。

5.4 综合讨论

综合以上两组实验的数据，本方案在 $k = 4$ 的配置下呈现出以下整体特征：

在良好哈希（随机均匀键）条件下， $n \leq 2 \times 10^5$ 范围内插入每键约 0.66–1.09 us（每次操作），查询每键约 0.24–0.43 us（每次操作），删除每键约 0.50–0.84 us（每次操作），内存每键约 44–46 B/key。与 `std::unordered_map` 相比，插入慢约 15–18 倍，查询慢约 20–25 倍，内存多约 1.4–1.7 倍。当 n 增至 10^6 时，合并操作导致的摊还成本使插入升至 7.47 us（每次操作）（约 12 倍于小规模），内存升至 102 B/key（约 3.2 倍于 `unordered_map`）。

然而，在极端碰撞条件下，两类方案的相对表现完全颠倒。当所有键被集中于同一个哈希桶时，`unordered_map` 的查询延迟随 n 线性增长（ $n = 10^5$ 时 132.5 us（每次操作）），而本方案保持在亚微秒级别（ $n = 10^5$ 时 0.39 us（每次操作）），速度优势约为 342 倍。这一反向关系的存在，意味着本方案的适用场景并

不在与标准库在常规负载下的绝对速度竞争，而在于那些对哈希碰撞具有高度敏感性的场景——例如开放哈希服务的安全防护、面对对抗性输入的数据库索引，以及需要严格最坏情况保证的实时系统。

第六章 LevelDB 集成

本章将实现的 k-kick 树哈希方案集成到 Google LevelDB^[2]的 MemTable 中，作为 SkipList 的点查询加速层。

6.1 集成流程与适配器设计

整体集成分为四个步骤：将四个头文件复制至 `leveldb/util/kick_hash/` 目录；编写适配层 `util/kick_hash.h` 完成 `Slice` 到 `uint64_t` 的类型映射；在 MemTable 的 Add 和 Get 路径中添加条件编译分支 `#ifdef USE_KICK_HASH`；修改 CMake 构建文件新增编译期开关。

适配层的核心是 `KickHash64` 哈希函数：调用两次 LevelDB 内置 `Hash()` 函数（不同种子），将两路 32-bit 结果拼接为 64-bit 哈希值，单次调用仅 5.17 ns。KickHashTable 包装器封装 Facility 对象，对外暴露 `Insert`、`Get` 和 `Remove` 三个接口。Facility 的 `lookup_value` 函数通过 `bool` 返回值替代异常，适配 LevelDB 的 `-fno-exceptions` 编译环境。

在 MemTable 层面，写入路径（Add）在执行原有 SkipList 插入后，通过 `kick_table_.Insert` 将 Arena 缓冲区指针同步存入 Facility；查询路径（Get）优先以 $O(1)$ 期望时间查询 KickHashTable，命中后解析 Arena 缓冲区并做原始 Slice 比对以防 64-bit 哈希碰撞（概率 $\sim 10^{-7}$ ），未命中则回退至 SkipList 的 $O(\log n)$ 查找。迭代器操作不受影响，始终基于 SkipList 的有序遍历能力。

6.2 双向开关设计

系统提供编译期和运行时两层开关。编译期通过 CMake 选项 `-DUSE_KICK_HASH=ON/OFF` 决定是否编译 KickHash 相关代码——关闭时无任何二进制膨胀或间接损耗。运行时通过 `SetUseKickHash(bool)` 接口支持同一进程内不同 MemTable 实例独立选择策略，便于 A/B 对比测试。

6.3 集成验证

以 `std::unordered_map` 和 `SkipList` 为对照，在 $n \leq 10^6$ 规模下使用 16 字节随机字符串键进行测试。核心结果见表6-1。

验证项目	测试条件	实测结果
KickHash64 调用延迟	单次调用	5.17 ns
插入延迟	$n = 2 \times 10^5$ ，16 字节随机键	0.86 us（每次操作）
查询延迟	$n = 2 \times 10^5$ ，16 字节随机键	0.35 us（每次操作）
删除延迟	$n = 2 \times 10^5$ ，16 字节随机键	0.56 us（每次操作）
全量查询正确率	$n = 10^6$ 次查询	100%
适配层开销占比	KickHash64/总插入延迟	< 0.15%

表 6-1 集成验证主要结果

适配层哈希开销在总体操作时间中占比不足 0.15%。在 $n = 2 \times 10^5$ 时查询延迟为 0.35 us（每次操作）， $n = 10^6$ 时约 0.74 us（每次操作），内存占用与第五章独立基准测试的测量数据一致（ $n = 2 \times 10^5$ 时 ~9 MB， $n = 10^6$ 时 ~97 MB）。全部 10^6 次查询正确率为 100%，未出现任何因 64-bit 哈希碰撞或 Arena 指针失效导致的误判。

第七章 总结与展望

7.1 工作总结

本文围绕 Bender 等人提出的 k -kick 树哈希方案^[7]，完成了从算法实现到系统集成的完整研究链条：

1. **核心算法实现**。以 C++20 完成了四层组件的 header-only 实现（总计约 1065 行）：MiniArray 位压缩数组（261 行）、LocalQueryRouter 紧凑二叉 Trie+LUT 路由（350 行）、Cubby 弹性哈希表含 free_bitmap_ 优先踢出链（419 行）、Facility 全局管理器含 deactivate 合并策略（233 行）。
2. **优先级踢出机制的精确对齐**。实现了 try_place 递归踢出链搜索、reserved 参数防回退、KickList 层级传递等关键细节，保证论文 §3.1^[7] 的优先级约束在代码层面逐条落地。
3. **多维度的定量实验**。以 std::unordered_map 为对照，在良好哈希 ($n \in [5 \times 10^4, 10^6]$) 和单桶碰撞 ($n \in [10^3, 10^5]$) 两种条件下完成插入、查询、删除和内存四项对比，每组 5 次独立试验。良好哈希条件下 $n = 2 \times 10^5$ 时插入约 1.09 us（每次操作）、查询约 0.43 us（每次操作）、空间约 44 B/key；deactivate 合并将合并成本限制在 $O(\text{merged_keys})$ 级。高碰撞实验中，Facility 在单桶冲突链长 10^5 条件下查询仅 0.39 us（每次操作），而 unordered_map 退化至 132.5 us（每次操作）（为 Facility 的 342 倍）。
4. **LevelDB 集成与双向开关**。以适配器模式将方案集成至 LevelDB MemTable，实现 KickHash64 类型映射（5.17 ns/op）、Add/Get 路径条件编译改造、以及编译期 CMake 开关与运行时 SetUseKickHash() 双层级开关。集成验证中 10^6 次查询正确率 100%，适配层开销 $< 0.15\%$ 。

7.2 不足与展望

- **Per-cubby 独立路由器**。当前依赖单一全局 `cubby_router_`，合并时需逐键更新。论文^[7]的 per-cubby 多路由器架构——每路由器仅处理约 $O(\log n / \log \log n)$ 个键——可将合并成本进一步压缩。
- **动态容量模型的常数调节**。当前 $C_j = K / (\log^{(j)} n)^2$ 中 $K \approx 409600$ 为固定值，应根据全局 n 自适应调节以提升大 n 下的空间利用率。
- **参数 k 的可配置化**。当前 k 由 `compute_kick_params` 自动推导（通常收敛于 $k \approx 4$ ），应提升为可配置参数，使用户按需调节时空权衡曲线。
- **多线程并发支持**。当前为单线程串行模型，但不同 bin 和 Cubby 之间具备天然的并行性，可探索细粒度锁或无锁实现。
- **LUT 位宽扩展**。当前 8-bit LUT（256 项，1 KB）可将查询深度缩减约 40%。扩展至 12-bit（4096 项）或 16-bit（65536 项）可进一步逼近完整 Method of Four Russians^[20] 的 $O(1)$ 查询复杂度。

Bender 等人提出的 k -kick 树哈希方案在理论上给出了哈希表时空权衡的完整曲线。本文实现了该方案的核心算法，通过多维度实验验证了其性能特征，并在 LevelDB MemTable 中验证了 k -kick 方案的工程可行性。所完成的 C++ 代码（约 1065 行，header-only）、实验数据以及与 LevelDB 的集成适配器可供后续相关研究参考。

参考文献

- [1] BLOOM B H. Space/Time Trade-offs in Hash Coding with Allowable Errors[J]. Communications of the ACM, 1970, 13(7): 422-426. DOI: 10.1145/362686.362692.
- [2] GHEMAWAT S, DEAN J. LevelDB: A Fast and Lightweight Key/Value Database Library[CP/OL]. 2011 [2025-12-01]. <https://github.com/google/leveldb>.
- [3] LUO C, CAREY M J. LSM-based Storage Techniques: A Survey[J]. The VLDB Journal, 2020, 29(1): 393-418. DOI: 10.1007/s00778-019-00555-y.
- [4] 申耀, 郝豫鹏, 王子京, 等. 基于非易失性内存的数据库关键技术研究综述[J]. 计算机科学, 2024: 1-35.
- [5] PAGH R. Low Redundancy in Static Dictionaries with Constant Query Time[J]. SIAM Journal on Computing, 2001, 31(2): 353-363. DOI: 10.1137/S0097539700369949.
- [6] FREDMAN M L, KOMLÓS J, SZEMERÉDI E. Storing a Sparse Table with $O(1)$ Worst Case Access Time[J]. Journal of the ACM (JACM), 1984, 31(3): 538-544. DOI: 10.1145/828.1884.
- [7] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[C]//Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC). ACM, 2022: 1284-1297. arXiv: 2111.00602 [cs.DS]. DOI: 10.1145/3519935.3519968.
- [8] LIM H, FAN B, ANDERSEN D G, et al. SILT: A Memory-Efficient, High-Performance Key-Value Store[C]//Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP). 2011: 1-13. DOI: 10.1145/2043556.2043558.

- [9] PAGH R, RODLER F F. Cuckoo Hashing[J]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2003.12.002.
- [10] KIRSCH A, MITZENMACHER M, WIEDER U. More Robust Hashing: Cuckoo Hashing with a Stash[C]//Lecture Notes in Computer Science: Proceedings of the 16th Annual European Symposium on Algorithms (ESA): vol. 5193. Springer, 2008: 611-622. DOI: 10.1007/978-3-540-87744-8_51.
- [11] ARBITMAN Y, NAOR M, SEGEV G. De-amortized Cuckoo Hashing: Provable Worst-case Performance and Experimental Results[C]//Lecture Notes in Computer Science: Proceedings of the 36th International Colloquium on Automata, Languages and Programming (ICALP): vol. 5555. Springer, 2009: 107-118. DOI: 10.1007/978-3-642-02927-1_11.
- [12] HERLIHY M, SHAVIT N, TZAFRIR M. Hopscotch Hashing[C]//Lecture Notes in Computer Science: Proceedings of the 22nd International Symposium on Distributed Computing (DISC): vol. 5218. Springer, 2008: 350-364. DOI: 10.1007/978-3-540-87779-0_24.
- [13] RAMAN R, RAO S S. Succinct Dynamic Dictionaries and Trees[C]//Lecture Notes in Computer Science: Proceedings of the 30th International Colloquium on Automata, Languages and Programming (ICALP): vol. 2719. Springer, 2003: 357-368. DOI: 10.1007/3-540-45061-0_30.
- [14] LIU M, YIN Y, YU H. Succinct Filters for Sets of Unknown Sizes[C]//Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC). 2020: 855-868. DOI: 10.1145/3357713.3384294.
- [15] DIETZFELBINGER M, MEYER AUF DER HEIDE F. A New Universal Class of Hash Functions and Dynamic Hashing in Real Time[C]//Lecture Notes in Computer Science: Proceedings of the 17th International Colloquium on Automata, Languages and Programming (ICALP): vol. 443. Springer, 1990: 268-283. DOI: 10.1007/BFb0032042.

- [16] CARTER J L, WEGMAN M N. Universal Classes of Hash Functions[J]. Journal of Computer and System Sciences, 1979, 18(2): 143-154. DOI: 10.1016/0022-0000(79)90044-8.
- [17] JR. G L S, LEA D, FLOOD C H. Fast Splittable Pseudorandom Number Generators[C]//Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA). ACM, 2014: 453-472. DOI: 10.1145/2660193.2660195.
- [18] BELAZZOUGUI D, BOTELHO F C, DIETZFELBINGER M. Hash, Displace, and Compress[C]//Lecture Notes in Computer Science: Proceedings of the 17th Annual European Symposium on Algorithms (ESA): vol. 5757. Springer, 2009: 682-693. DOI: 10.1007/978-3-642-04128-0_61.
- [19] CHANG F, DEAN J, GHEMAWAT S, et al. Bigtable: A Distributed Storage System for Structured Data[C]//Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, 2006: 205-218. DOI: 10.1145/1365815.1365816.
- [20] ARLAZAROV V L, DINIC E A, KRONROD M A, et al. On an Economical Construction of the Transitive Closure of a Directed Graph[J]. Doklady Akademii Nauk, 1970, 194(3): 487-488.

致 谢

转眼间，校园生活即将结束。此篇论文完稿之际，要感谢众多师长和亲友，谢谢你们期望与鼓励。此时此刻，我无法找到合适的言语来表达我内心深处最真挚的谢意。在本毕业设计的选题、研究与撰写过程中，笔者得到了多方面的帮助与支持

首先衷心感谢指导教师在论文选题、研究方法、实验设计以及论文撰写等各个阶段给予的悉心指导和宝贵建议。

然后感谢同窗的各位同学。你们为我在论文理论分析和实验过程中提供了大量的无私帮助，这份同窗之情将是最值得留恋的回忆。

我还要感谢我的父母，谢谢他们默默的支持。

另外感谢南京大学 Linux 用户组 (NJU LUG) 提供的 NJUThesis 毕业论文模板，其规范化的 $\text{\LaTeX}$ 文档结构和精美排版极大地减轻了论文格式调整的工作负担。

感谢 Google LevelDB 项目^[2]的全体开发者贡献了一个结构清晰、代码规范、文档完善的键值存储引擎实现，使得本文的数据库集成方案能够在一个成熟、稳定且具有广泛影响力的开源平台上得到验证和展示。

最后，还要感谢在 github 上为我解答了不少疑问的开发者们。